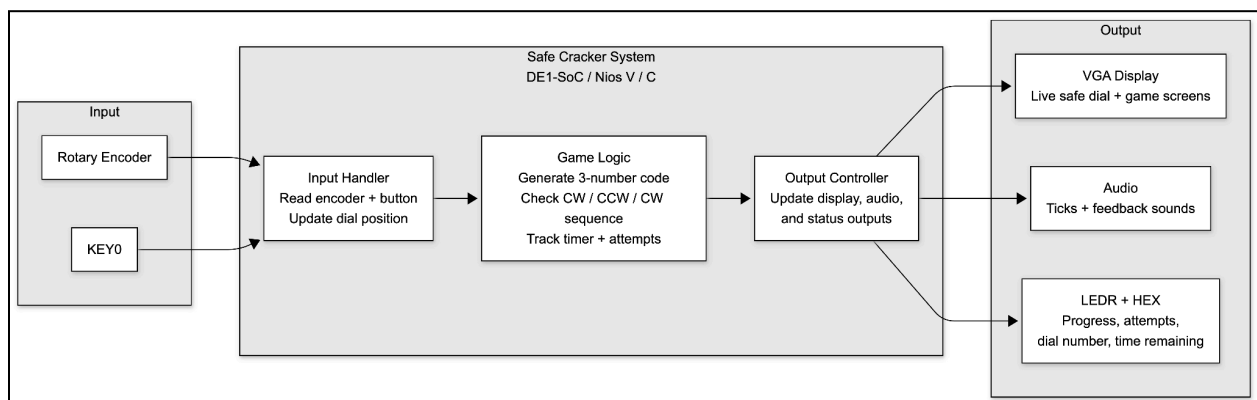


1.0 Summary

Safe Cracker is a timed game that simulates opening a rotational safe lock on the DE1-SoC. The system generates a hidden 3-number combination, and the player uses a rotary encoder to turn an on-screen safe dial left or right. As the dial moves, audio cues help the player identify important dial positions. The player presses KEY0 to submit the current dial number as a guess. To crack the safe, the player must correctly enter all 3 numbers in order, using the same pattern as a real lock: first number clockwise, second number counterclockwise, and third number clockwise. Each correct guess advances the player to the next stage of the lock and is shown using LEDs, audio, and the VGA display. An incorrect guess resets lock progress, and after 3 incorrect guesses, the player loses. The player also loses if the timer expires before the full combination is entered. If all 3 numbers are entered correctly before time runs out, the safe is unlocked and the game is won.

Optional: Incorporate multiple stages; clearing 1 safe adds a score value.

2.0 Block Diagram



3.0 Division of Work

Week/Milestone	Work	Division of Work
Week 1 March 20	Setup rotary encoder through interrupts to test functionality. Setup VGA with basic static visuals on the screen.	Tej: Setup the rotary encoder to detect clockwise and counterclockwise rotation through interrupts Leo: Set up VGA output and create a basic static game screen to confirm that visuals display properly on the monitor.
Week 2 March 27	Complete most of the game/backend logic along with visuals.	Tej: Complete game logic: matching rotation to random codes, correct/incorrect states, start/end state, timer, KEY0 to confirm. Help out with VGA if needed

		Leo: Complete the main visual system, including drawing the safe dial, using double buffering for dial rotation animation, and creating the gameplay, success, and game over screens.
Week 3 April 3	Merge VGA and game logic (if not already completed). Integrate audio, LEDs, and other optional hardware devices (see above).	Work together to integrate VGA and game logic, implement audio, and any additional hardware devices if time permits.
Before Presentation April 7	Debug, create the report, and plan demo.	Work together to solve any bugs, create the final report, and practice our demo.

4.0 Program Features

Input

- Rotary encoder from parallel ports - detect clockwise or counterclockwise turns
- KEY0 to submit the current dial position as a guess
- *Optional: microphone input to detect loud user sound; if the sound exceeds a threshold, the screen turns red and user input is temporarily blocked*
- *Optional: Use an accelerometer to detect user motion -> if the user is moving too much while trying to spin the dial, screen turns red*

Output

- VGA display showing the safe dial rotating to different numbers based on rotary encoder movement
- VGA game states for active gameplay, locked out/game over, and game complete
- Audio for dial movement ticks, correct-position special tick sound, correct guess, and incorrect guess
- LEDR0-2 = indicate progress through the 3-number lock sequence (1, 2, 3)
- LEDR3-5 = attempts remaining
- HEX0-1 = current dial number
- HEX2-3 = time remaining
- Optional : HEX4-5 = score, where score is the number of safes opened

Internal Logic

- Map rotary encoder movement to a fixed range of dial values on the safe
- Randomly generate a 3-number lock code at the start of each game

- Track the current stage of the lock sequence, remaining time, and number of incorrect guesses
- Animate the safe dial turning left or right based on encoder input
 - Play a tick sound whenever the dial passes a value
 - Play a distinct audio cue when the dial reaches the correct value for the current stage of the lock
- **Screen states**
 - **Game loop:** Animate the lock and have the user turn it using the rotary encoder
 - **Locked out:** Player fails to crack the lock within the required time or reaches 3 incorrect guesses -> screen turns red
 - **Game complete:** player correctly enters all 3 numbers before time expires -> screen turns green and congratulates them
- **Game Logic FSM**
 - Randomly generate 3 numbers: these are the numbers the user will rotate to crack the code
 - User turns rotary encoder to a number, press KEY0 to confirm
 - If it's correct, progress LEDR will light up and sound plays, advance to next lock stage
 - If it's incorrect, different audio plays to indicate the user is incorrect. Increment incorrect guess counter, reset lock progress to the first number, reset progress LEDR
 - User keeps turning and guessing. They have 3 guesses to guess the code, if they use up all 3 and can't get the code, screen turns red
 - Like a real lock, user must guess first number turning clockwise, second number turning counter clockwise, and third number turning clockwise
 - If all 3 numbers are entered correctly, transition to Game Complete
 - If time runs out or the player reaches 3 incorrect guesses, transition to Locked Out / Game Over